

SFA – Delivered Milestones and Current Status

Overview

SmallFarmsAgents completed **9 milestones** (M1-M9) of active development before entering its AOS canonization phase (S001). Every milestone passed a formal quality gate – a three-phase process of implementation, independent QA validation, and documentation. This document details what each milestone delivered and the current state of the live system.

Milestone Development Model

Each milestone followed a three-phase gate model:

- **Phase A – Implementation:** The implementing team builds the feature set and unit tests
- **Phase B – QA Validation:** Team 50 (QA) independently validates against the milestone specification – integration tests, data quality checks, regression tests, and end-to-end pipeline runs
- **Gate G[?] – Sign-off:** Written gate pass required before the next milestone begins. No team advances without a signed gate

This model – separating implementation from validation – is borrowed from AOS's constitutional governance and applied throughout SFA's development.

M1 – Local Foundation (G1: PASS)

Team: Team 20 (Infrastructure)

The foundation milestone established the entire data layer before any collection or normalization code was written:

- Python 3.11+ project skeleton – `organic_market_agent/` package with all submodule structure
- PostgreSQL 15 database via Docker – 23 initial tables + 2 views

- SQLAlchemy 2.x ORM models for all tables
- Alembic migration system – 5 initial revisions covering schema, seed data
- Seed data: 11 measurement units, 4 unit conversions, 29 initial products, 20 data sources
- Utility layer: logging, config management, checksum utilities
- CLI health check: `python -m organic_market_agent.db.check`
- 7 database health tests passing

Gate G1 acceptance criteria included: all 29 products present, all price columns

`NUMERIC(12,4)` (no FLOAT anywhere), all timestamp columns `TIMESTAMPTZ`, migration round-trip (downgrade base → upgrade head) verified.

M2 – Collection Layer (G2: PASS)

Team: Team 10 (Feature Dev)

The collection milestone built the HTTP fetch engine that retrieves raw data from organic farm websites and APIs:

- `CollectorEngine` with `BaseCollector` – retry logic, exponential backoff, timeout handling, checksum deduplication
- Three collector implementations:
 - `EasyFarmCollector` – for farms hosted on the EasyFarm platform (API responses)
 - `StandaloneHTMLCollector` – for farm websites with custom HTML layouts
 - `GovtBenchmarkCollector` – for government reference pricing APIs
- `ParserEngine` dispatcher with three parsers:
 - `EasyFarmCatalogParser` – processes EasyFarm JSON API responses
 - `SimpleProductGridParser` – extracts products from HTML grid layouts
 - `OfficialWholesaleParser` – processes government reference data
- `IngestionRunner` CLI entry point
- Raw HTTP responses stored as files; metadata tracked in `raw_assets` table
- `raw_extracted_items` populated with structured raw data

Gate G2 verified: ≥3 live sources collected, ≥50 raw extracted items, deduplication confirmed (re-run → zero new `raw_assets`), failed-source error handling verified.

M3 – Normalizer Engine (G3: PASS)

Team: Team 10

The normalizer is the intellectual core of SFA – the component that converts raw, inconsistent Hebrew product names and prices into a canonical, comparable dataset.

- 8-stage normalization pipeline (see SFA_02 for full stage breakdown)
- Data-driven design: all normalization logic stored in PostgreSQL – alias mappings, scope-skip rules, unit conversions – configurable without code changes
- Alias system: maps raw Hebrew product names to canonical catalog products
- Scope-skip system: filters non-food and out-of-scope items using configurable pattern rules
- Organic flag detection
- Price parsing with range handling (e.g., "₪15-20" → midpoint)
- Unit resolution and quantity extraction
- Price normalization to ₪/kg equivalents
- Confidence scoring for each normalized observation

Gate G3 included: a formal Data Quality and Cohort Gate Specification

(`M3_DATA_QUALITY_AND_COHORT_GATE_SPEC.md`) – establishing the phased lifecycle spec for how data quality would be measured throughout the pipeline going forward.

M4 – Aggregator (G4: PASS)

Team: Team 10

The aggregator computes daily statistics from normalized observations and applies quality gates before marking data as publishable:

- Daily aggregation per product: mean price, standard deviation, observation count, count by source
- Weekly snapshot generation for trend tracking
- QA Engine with two outlier detection rules:

- **2-source spread rule:** If only 2 sources and `|price1 - price2| > 50%` of lower price → flagged
- **Multi-source σ rule:** If $\sigma > \approx 2/\text{kg}$ across all observations → flagged for admin review
- Minimum source requirement: ≥ 2 community sources must contribute before a price is published
- Staleness computation: `ok` (current) / `warning` (3+ days) / `stale` (8+ days)
- Observation flagging system: admin can mark individual observations as `hide` or `review`

Gate G4 required a waiver: Gate G4 was resolved with a formal QA-001 Waiver for a specific data quality condition, documented in `_COMMUNICATION/team_100/reports/2026-03-30_ARCH_DECISION_G4_QA001_WAIVER_TEAM100.md`. This demonstrates the governance rigor: known quality issues are documented and waived explicitly rather than silently ignored.

M5 – Admin Interface (G5: PASS)

Team: Team 10

The admin UI is a Flask web application (Hebrew RTL, port 5001) that gives the operator full visibility into the pipeline:

- **Dashboard** – pipeline status, today's run summary, recent runs, alerts, resolution rate funnel
- **Sources** – source registry management, fetch profile editing, active/inactive toggle
- **Products** – canonical product catalog browser, display settings, alias coverage view
- **Aliases** – alias management (raw Hebrew name → canonical product mapping)
- **Scope Skip Rules** – manage the 301 scope-skip rules
- **Observations** – browse normalized observations, apply hide/review flags, inspect unresolvable items
- **Aggregates** – daily and weekly statistics per product
- **Runs** – pipeline execution history, per-run reports
- **Alerts** – in-app pipeline and operational alerts
- **Publish** – trigger manual publish runs, artifact history

The admin UI is for the operator only – it is not the public-facing product. It runs on localhost (`127.0.0.1:5001`) and is the control plane for all pipeline operations.

Test scope at G5: 25+ Flask templates, browser-based functional UI flows tested by Team 50.

M6 – Automated Scheduler (G6: PASS)

Team: Team 10

The automation milestone wired the pipeline into a production-capable scheduler:

- System cron → Python runner integration
- **Self-gating pipeline:** Each stage checks its prerequisites before running – if no new raw assets, normalization is skipped; if normalization produced nothing new, aggregation is skipped
- Per-source scheduling: each source has its own cron expression in its fetch profile
- Retry and backoff logic for transient failures
- Log entries written to the database for every run
- Run reports accessible via the admin UI

Operational constraint: The scheduler is production-only. On developer machines, all pipeline stages are manual-only by policy – to prevent duplicate production artifacts from being uploaded.

M7 – Public Publishing (G7: PASS)

Team: Team 10 / Team 100

The publishing milestone delivered the public artifact system and WordPress integration:

- `PublishEngine` building versioned JSON + HTML artifacts
- `manifest.json` with `schema_version`, `artifact_version`, staleness metadata, fixed-name pointers
- `manifest_last_good.json` fallback for resilience
- Custom `ReusedSessionFTP_TLS` FTPS client for uPress compatibility

- FTPS upload to `wp-content/uploads/market/` on uPress server s887
- ezCache WordPress cache invalidation after upload
- Public manifest verification (HEAD request) after upload
- WordPress shortcode integration: `[sfagent_market_report]`

G7 was the first gate requiring **Nimrod sign-off** (human approval) in addition to Team 50 QA – because this milestone crossed the boundary from private development to public production.

M8 – UX Polish + Policy Formalization (G8: PASS)

Team: Team 10, Team 80

With the pipeline live, M8 focused on improving the public-facing experience:

- **Tooltip layer** for statistical terms – all column headers in the price table have Hebrew explanations (hover on desktop, tap on mobile), custom JS, no external dependencies
 - **Community CTA banner** – below the price table, above the transparency block, with WhatsApp link for data submission
 - **Visual hierarchy enhancement** – average price column is the dominant visual element; median and range are subordinate; stddev hidden on mobile
 - **SEO and meta data** – WordPress page title, meta description, Hebrew keywords, canonical URL
 - **Privacy Policy** – public policy document for data collection and publication
 - **RTL Development Guide** – documented RTL conventions for all future frontend work
-

M9 – Resolution Rate and Site Optimization (G9: PASS)

Team: Team 10, Team 80

The resolution rate milestone targeted 100% normalization of all extractable items:

- **Resolution rate achieved: 100%** – 0 unresolvable items out of all extracted items
- Expanded alias library to 232 mappings covering all active source product names
- Expanded scope-skip rules to 301 patterns covering all out-of-scope items

- Site optimization: page performance, CSS scoping, responsive layout refinement
- Data quality snapshot embedded in public_report.json and manifest.json – consumers can see pipeline health metrics

100% resolution rate is the key quality achievement. It means every product name SFA encounters on active sources is either successfully mapped to a canonical product or explicitly classified as out-of-scope. Nothing falls through without a deliberate classification.

S001 – AOS Canonization (COMPLETE, 2026-04-12)

After M9, SFA was formally adopted into the AOS governance framework:

- `_aos/` governance structure created: `definition.yaml`, `roadmap.yaml`, `governance/`, `lean-kit/`
 - `CLAUDE.md` written and committed
 - Lean Kit v3.1.7 snapshot deployed
 - Hub registration in agents-os
 - `validate_aos.sh` 26 PASS / 9 SKIP / 0 FAIL at canonization
 - L-GATE_V PASS by Team 90 (cross-engine constitutional validation)
-

Current System State (April 2026)

Metric	Value
Data sources registered	20 (7 currently active)
Canonical products	67 (62 with observations)
Product alias mappings	232
Scope-skip rules	301
Normalized observations	174
Resolution rate	100%
Alembic migrations	31 (head: 031)
Automated tests	127 passing, 2 skipped
Milestones complete	M1-M9 + S001
Production deployment	waldhomeserver (Ubuntu 24.04)
Public URL	nimrod.bio/smallfarmsagent
Daily pipeline	Running (cron → waldhomeserver)

What 100% Resolution Rate Means

The resolution rate metric deserves specific attention because it is not a trivial achievement.

When SFA encounters a product name like “כרוב ירוק קטן בשלפוחית” (small green cabbage in a bag) on a farm website, it must decide: is this a canonical product? Which one? What unit? What quantity? If it cannot answer these questions, the item is marked `unresolvable` – it exists in the database, is flagged for admin review, but does not appear in the published index.

Reaching 100% resolution means building enough alias coverage that every product name on every active source can be classified. This required:

- Curating 232 alias mappings through iterative pipeline runs – each time an unresolvable item appeared, the admin reviewed it, decided whether it belonged in scope, and either added an alias or added a scope-skip rule

- 301 scope-skip rules developed by examining every non-food item that appeared on farm websites (cleaning products, donations, dry grocery, packaged imported goods)

The 100% resolution rate is a living metric. Every time a new source is added, or an existing source changes its product names, the rate can temporarily drop – and the admin review queue fills with new unresolvable items to classify.